



Python 基本语法



1 保留字



1 保留字

保留字是Python语言中一些已经被赋予特定意义的单词，这就要求开发者在开发程序时，不能用这些保留字作为标识符给变量、函数、类、模板以及其他对象命名。

```
import math
a, b, c = map(int, input().split())
global x
x = 0
for i in range(10):
    if i % 2 == 0:
        print(str(a) * i)
    elif i % 3 == 0:
        print(str(b) * i)
        break
    else:
        x += 1
print(x)
```

1 保留字



➤ Python中的所有保留字:

All keywords in Python :

```
>>> import keyword
>>> print(keyword.kwlist)
['False', 'None', 'True', 'and', 'as',
'assert', 'async', 'await', 'break', 'class',
'continue', 'def', 'del', 'elif', 'else',
'except', 'finally', 'for', 'from', 'global',
'if', 'import', 'in', 'is', 'lambda',
'nonlocal', 'not', 'or', 'pass', 'raise',
'return', 'try', 'while', 'with', 'yield']
```



02 基本数据类型

2 基本数据类型和变量:



- 2.1、数字类型: 包括整型(int)和浮点型(float)
- 2.2、字符串类型: str
- 2.3、布尔类型: 包括真(True)和假(False)
- 2.4、空类型: None
- 2.5、组合数据类型: 包括列表(list)、元组(tuple)、集合(set)和字典(dict)

2.1.1 数字类型



- 整型(int): 整数类型
- 浮点型(float): 小数类型
- 整型与浮点型的强制转换: 用int(数字)或float(数字)

int(浮点型): 去尾

float():

1、整型: 保留一位小数

2、浮点型: 不变

3、末尾有多余的0: 去掉所有的0后再处理

```
>>> print(int(3.14159265))
3
>>> print(int(3.99999999999))
3
>>> print(float(4))
4.0
>>> print(float(4.000000000))
4.0
>>> print(float(4.1212000000000))
4.1212
```



2.1.2 数字类型的运算符

- “+”：用于两个数字类型相加。
- “-”：用于两个数字类型相减。
- “*”：用于两个数字类型相乘。
- “/”：用于两个数字类型相除，其结果必为浮点型。
- “//”：用于两个数字类型相整除（向下取整），其结果必为整型。
- “%”：用于两个数字类型取余，如 $8\%3=2$ 。
- 不同的数字类型运算，浮点型会覆盖整型。如 $100.12 \times 200 = 20024.0$ 。

2.2.1 字符串的类型



➤ 表示方法: ” 字符串内容”

```
>>> print(type(12345))  
<class 'int'>  
>>> print(type("12345"))  
<class 'str'>
```

➤ 字符串中操作符的使用: (“+” 和 “*”)

```
>>> print("abcde"+"fghi")  
abcdefghi  
>>> print("abcde"*3)  
abcdeabcdeabcde
```

2.2.2 字符串的操作函数



➤ 1. lower() 和 upper()：用于把字符串变为全部大写或小写

```
>>> print("FBASIhuidhasiibFHUIADBU2138hduq,,909..".upper())  
FBASIHUIDHASIIBFHUIADBU2138HDUQ,,909..  
>>> print("FBASIhuidhasiibFHUIADBU2138hduq,,909..".lower())  
fbasihuidhasiibfhuiadbu2138hduq,,909..
```

注意函数的用法！

upper()：把字符串所有的小写字母变成大写，其它不变

lower()：把字符串所有的大写字母变成小写，其他不变

2.2.2 字符串的操作函数



➤ 2、split(sep): 根据sep分割字符串，返回一个列表（下一章讲）

```
>>> print("abcabcabcabc".split("bc"))
['a', 'a', 'a', 'a', 'a', '']
>>> print("a,b,c,d,e".split(","))
['a', 'b', 'c', 'd', 'e']
>>> print("a b c d e".split())
['a', 'b', 'c', 'd', 'e']
>>> print("a b c d e ".split())
['a', 'b', 'c', 'd', 'e']
>>> print("a,,b,,c,,,d,,,e,,,".split(","))
['a', '', '', 'b', '', 'c', '', '', 'd', '', '', 'e', '', '', '']
```

2.2.2 字符串的操作函数



➤ 2、`split(sep)`：根据`sep`分割字符串，返回一个列表（下一章讲）

用法：

(1) 将`sep`变为一个空格字符（“ ”），在列表中时，若该空格字符后面有非空格字符，则删去该空格字符。

(2) 若`sep`为“ ”，则保证分割后的列表内不会出现“ ”。

(3) 若`sep`为`None`（或空），则默认为“ ”。

拓展思考：如何把`split()`函数用代码实现？

2.2.2 字符串的操作函数



- 3、count(str): 返回str在字符串中出现的次数

```
>>> print("An apple a day!".count("a"))  
3
```

- 4、replace(old, new): 把原str的old替换为new后返回一个新str

```
>>> print("abcdefgabcdefg".replace("a", "aaaaaa"))  
aaaaaabcdefgaaaaaabcdefg  
>>> print("aaaaabbbcccccaa".replace("aa", "g"))  
ggabbbccccga
```

2.2.2 字符串的操作函数



➤ 5、strip(sep): 从字符串左侧及右侧去掉sep中出现的字符后返回一个新的字符串。

步骤:

1、从左侧开始，看第一个字符是否在sep中出现，若是，则删除它，然后看第二个字符，继续执行步骤，直到有一个字符不在sep中出现为止。

2、从右侧向左，继续执行1的步骤。

```
>>> print("abdcefgabcdefg".strip("abfg"))
dcefgabcde
>>> print("  == I love python ==  ".strip(" =n"))
I love pytho
```

2.2.2 字符串的操作函数



- 6、sep.join(str): 在str除最后一个元素外的每个元素后面插入sep, 然后返回一个新的字符串。

```
>>> print(", ".join("ABCDEFGH"))  
A, B, C, D, E, F, G  
>>> print(".".join("I love python!  "))  
I. .l.o.v.e. .p.y.t.h.o.n!. .
```

2.2.2 字符串的操作函数



➤ 7、str与int/float的转换。

用str()将int转换为str，用int()将str转换为int。（float同理）

将str转换为int时，str必须为纯数字。

此时int函数可以进行进制转换：int(str, base)

将str从base进制转换为10进制，base不写默认为10

```
>>> print(int("101010101",2))  
341  
>>> print(int("a8b700",16))  
11056896  
>>> print(int("12345678",9))  
6053444
```


2.3 布尔类型



➤ 只有真(True)和假(False)，注意大写。其中真为1，假为0

➤ 布尔类型的运算符：and, or, not

and: 只要有一个变量是0，结果为0

or: 只要有一个变量是1，结果为1

not: 1的结果为0，0的结果为1

➤ 优先级: not > or > and

➤ 思考: print(True and False or True)的结果?

2.4 空类型



➤ 用None表示，它表示什么都没有，和0是不同的。

```
>>> print("a b c d e".split())  
['a', 'b', 'c', 'd', 'e']  
>>> print("a b c d e".split(None))  
['a', 'b', 'c', 'd', 'e']
```

2.5.1 列表



- 列表类型类似于C语言里的数组，它的定义方法为：列表名称 = [<元素>]，例如List = [1, 2, 3, 4, 5]。其中1, 2, 3, 4, 5分别是列表中的第0、1、2、3、4个元素，可以用下标运算符[]获取List中的元素

```
List = [1, 2, 3, 4, 5]
print(List[2]) # 输出3
List[0] = 0 # 将第0个元素（即1）替换为0
```

- 创建空列表，如：List(列表名称) = []
- Python中的组合数据类型实际上是广义表，每个元素的类型可以不一样，可以是int, float, str, 甚至list, tuple, set, dict等

2.5.1 列表



➤ 字符串转化为列表

使用list()函数，可以将str类型，tuple类型，set类型转换为list类型

```
string = "I Love Python!"  
List = list(string)  
print(List)
```

```
['I', ' ', 'L', 'o', 'v', 'e', ' ', 'P', 'y', 't', 'h', 'o', 'n', '!']
```

进程已结束,退出代码0



2.5.1 列表

➤ 列表的一些操作及方法

1、List[i]: 取第i个元素

```
List = [1, 2, 3, 4, 5]
print("List[0] = {}".format(List[0]))
print("List[4] = {}".format(List[4]))
print("List[-1] = {}".format(List[-1]))
print("List[-5] = {}".format(List[-5]))
```

```
List[0] = 1
List[4] = 5
List[-1] = 5
List[-5] = 1
```

进程已结束,退出代码0



2.5.1 列表

➤ 列表的一些操作及方法

2、List[i:j:k]: 列表切片，返回一个列表，从i到j（不包括j，左闭右开）以k为步长，i不写默认为0，j不写默认为len(List)，k不写默认为1

```
List = [1, 2, 3, 4, 5]
print("List[1:3] = {}".format(List[1:3]))
print("List[:2] = {}".format(List[:2]))
print("List[1:4:2] = {}".format(List[1:4:2]))
print("List[-1:-3:-1] = {}".format(List[-1:-3:-1]))
```

```
List[1:3] = [2, 3]
List[:2] = [1, 2]
List[1:4:2] = [2, 4]
List[-1:-3:-1] = [5, 4]
```



2.5.1 列表

➤ 列表的一些操作及方法

函数或方法	描述
List1 += List2	将List2中的元素插入List1的后面
List1.append(x)	将x插入到List1的最后一个元素后面
List1.insert(i, x)	将x插入到List1的第i个元素前面
List1.remove(x)	将List1中第一个出现的x删除
List1.pop(i)	将List1中的第i个元素删除
List1.clear()	清空List1中的元素
List1.reverse()	反转List1中的元素

2.5.3 集合



- 元组类型与列表类型的唯一区别就是：元组类型一旦被定义，就不可以进行元素改变（例如插入、删除、修改等），它的定义方式为：元组名称 = (<元素>）。

```
Tuple = (1, 2, 3, 4, 5)
```

```
Tuple[3] = 4
```

| 元组不支持项目赋值

- 创建空元组，如：Tuple(元组名称) = ()
- 使用tuple()函数可以将list/str/set类型转换为tuple类型

➤ 元组的一些操作及方法

函数或方法	描述
<code>Tuple[i:j:k]</code>	返回一个元组（元组的切片操作，与列表一样）
<code>len(Tuple)</code>	返回Tuple的长度（即元素个数）
<code>min(Tuple)</code>	返回Tuple中的最小值（需要可比较）
<code>max(Tuple)</code>	返回Tuple中的最大值（需要可比较）
<code>Tuple.index(x)</code>	返回Tuple中第一次出现x的下标值
<code>Tuple.index(x, i, j)</code>	返回Tuple中从i到j中第一次出现x的下标值
<code>Tuple.count(x)</code>	返回Tuple中x出现的次数

➤ 注：这些函数在list类型中同样适用

2.5.3 集合



- 集合类型与列表类型的唯一区别就是：集合里的元素具有互异性，即不会出现相同的元素，定义方法为：
集合名称 = {<元素>}；
- 创建空集合的方法为：Set(集合名称) = set()
- 可以用set()函数将list/str/tuple类型转化为set类型

```
List = [1, 1, 2, 2, 3, 3, 4, 5, 1, 6, 3, 6]  
print(set(List))
```

```
{1, 2, 3, 4, 5, 6}
```

进程已结束,退出代码0

注意：使用set()函数后可能会打乱原来的顺序

➤ 集合的一些操作及方法

函数或方法	描述
<code>Set1 & Set2</code> (<code> </code> <code>-</code> <code>^</code>)	集合的交/并/差/补运算，会返回一个新的集合
<code>Set.add(x)</code>	如果x不在集合中，将x加入集合中
<code>Set.discard(x)</code>	移除集合中的元素x，如果集合中不含该元素不报错
<code>Set.pop()</code>	随机移除集合中的一个元素并返回，集合不能为空
<code>x in Set</code>	若x在集合中返回True，否则返回False
<code>x not in Set</code>	若x不在集合中返回True，否则返回False

2.5.4 字典



- 类型中的每一个元素都由一对键值对构成，每一对键值对由一个键和一个值构成。如Dict = { “键1” : “值1” , “键2” : “值2” , }
- 创建一个空字典的方法: Dict = {}

```
D = { 'a': 'Alice', 'b': 'Bob', 'c': 'Cindy' }  
print(D['a'])
```

Alice

进程已结束,退出代码0

➤ 字典的一些操作及方法

函数或方法	描述
Dict[key] = value	在字典中插入键值对，其中键为key，值为value
Dict.keys()	以列表形式返回字典中所有的键
Dict.values()	以列表形式返回字典中所有的值
Dict.items()	以列表形式返回字典中所有的键值对
Dict.pop(x, <d>)	若键x存在，取出相应的值，否则返回<d>
Dict.get(x, <d>)	若键x存在，返回相应的值，否则返回<d>
Dict.popitem()	随机从字典中取出任意一个键值对，以元组形式返回



03 变量

3.1 变量概述



- 变量可以是任何类型，Python中的变量类型是动态的，即不需要声明其类型，但是如果在某些条件下由于类型没有声明而造成TypeError，这时就需要用函数来对其声明类型。

如：

`int(x)`：把x转换为int型；

`str(x)`：把x转换为str型；

.....

以下对变量的赋值是符合规则的：

```
a = 1 # int类型
b = 0.12345 # float类型
c = "python" # str类型
d = [1, 2, 3, 4] # list类型

b = str(b) # 将b转化为str类型
d = set(d) # 将d转化为set类型
```

3.2 变量的命名



➤ 变量的命名规则为：

(1) **必须以字母、下划线开头**，后面可以跟任意数目的字母、数字和下划线。此处的字母并不局限于 26 个英文字母，可以包含中文字符、日文字符等。

(2) 由于 Python 3 支持 UTF-8 字符集，因此 Python 3 的标识符可以使用 UTF-8 所能表示的多种语言的字符。Python 语言是**区分大小写的**，因此 abc 和 Abc 是两个不同的标识符。

(3) 命名时**不能使用保留字**。

3.2 变量的命名



➤ 判断下列变量的命名是否正确：

- | | |
|----------------|-------------------------|
| (1) true | 正确（注意保留字是True，首字母大写） |
| (2) 123abc | 错误，不能以数字开头 |
| (3) list_a | 正确 |
| (4) _self_ | 正确 |
| (5) odd number | 错误，不能带有除下划线的其他字符（此处是空格） |
| (6) await | 错误，这是保留字 |

3.2 变量的命名



➤ 希望大家养成好的命名习惯：

(1) 你的变量名字应该有一定意义：

如count表示计数器，sum表示总和，number表示一个数字

当变量多时，如果你仅仅用a, b, c, d, e, ...来表示变量，会很难理解一段代码

(2) 拼音的理解速度会比英文慢，所以最好用英语来命名。当一个名字需要两个以上单词组合时，有两种命名方法（以正方形面积为例）：

①驼峰命名法：首字母大写。如SquareArea.

②下划线分隔法：用下划线分隔。如square_area.

3.3 注释



- 在写代码时，有时为了解释一些东西（比如你写的一长串判断语句是为了判断什么，或者写了一个函数是什么功能，或者定义了一个变量是用来干嘛的），可以在其之后写注释。只要在注释语句前加入#号就可。编译时不会编译#后的句子。

单行注释：在注释前 #，在代码中使用#时，右边的任何数据都会被忽略

```
print (" hello word ") #输出" hello word"
```

以下对变量的赋值是符合规则的：

```
a = 1 # int类型  
b = 0.12345 # float类型  
c = "python" # str类型  
d = [1, 2, 3, 4] # list类型
```

```
b = str(b) # 将b转化为str类型  
d = set(d) # 将d转化为set类型
```

3.3 注释



➤ 多行注释：在注释前后加上 ' ' ' （三个单引号）

' ' '

三对单引号，python多行注释

三对单引号，python多行注释

' ' '

希望大家养成多写注释的好习惯



04 输入和输出

4.1 基本输入



输入: `input()`, 从键盘上读取, 遇到回车时结束

输出: `print()`, 将括号内的内容输出

```
>>> print(123)
```

```
123
```

正确, 当括号内为纯数字时, `print()` 会把它视为一个数字直接输出

```
>>> print("abc")
```

```
abc
```

正确, 双引号表示括号内是一个字符串, `print()` 也会直接输出

```
>>> print(abc)
```

```
Traceback (most recent call last):
```

```
File "<pyshell#4>", line 1, in <module>
```

```
    print(abc)      错误, 当括号内有字母时, print() 会把它视为一个变量, 必须赋值才能输出
```

```
NameError: name 'abc' is not defined
```

```
>>> a = 123
```

```
>>> print(a)
```

```
123
```

正确, `a` 是一个变量, 对变量赋值后可以输出变量的值

4.1 基本输入



注意：python3中，input()会把读取到的内容转化为字符串！

```
a = input()
print(type(a)) # 输出a的类型
```

123

<class 'str'>

表明123是一个字符串而不是一个数字

如果想把123转化为数字，要用类型转换函数int()

```
a = int(input()) # 将input()读取到的内容转换为int型
print(type(a)) # 输出a的类型
```

123

<class 'int'>

4.1 基本输入



- 在python3版本中，input() 从键盘上读取到的内容全部返回为str

```
a = input()
print(type(a)) # 输出a的类型
```

```
12345
<class 'str'>
```

- 可以使用类型转换函数

```
a = input()
b = input()
print(a + b)
```

```
12
23
1223
```

```
a = int(input())
b = int(input())
print(a + b)
```

```
12
23
35
```


4.1 基本输入



➤ 问题：假如输入一行数据“1 2 3 4 5”，分别赋值给变量a, b, c, d, e，数据以空格分隔，应该怎么赋值呢？

```
a, b, c, d, e = input().split() #分隔成5个元素，按顺序赋值
```

➤ 问题：假如要把元素都变成int类型呢？

```
a, b, c, d, e = map(int, input().split()) #map函数 后面讲
```

map(f, list)表示把函数f作用于list中的每个元素

4.2 基本输出



- 例子：我们都知道`print()`语句执行完之后会多打印一个换行，比如

```
print(123)      123
print(1234)     1234
print(12345)    12345
```

- 现在想把这三个数字放同一行，末尾不是换行而是一个空格，怎么实现？

```
print(123, end=' ')
print(1234, end=' ')
print(12345, end=' ')    123 1234 12345
```

- `print(..., end = k)`：输出时以`k`结尾，不写则默认为换行

简单的练习

➤ 从键盘输入两个数，求他们的和并输出

提示：把输入内容转换为数字（int型）

➤ 从键盘输入三个数到a, b, c中，按公式值输出

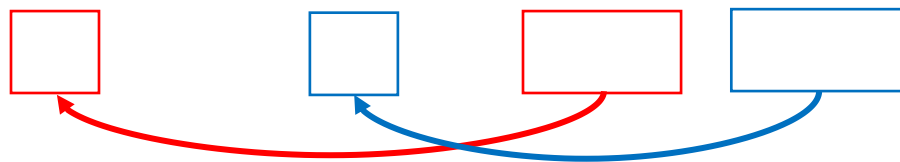
公式： $a = b * b + c * c * c$

4.3 格式化输出



- 例子：现在有一个数据库，存有每个学生的姓名(name)的成绩(score)，现在要求从键盘上输入一个学生的姓名，按” (姓名)的成绩是 (成绩) “的语句来输出，应该怎么用一句print语句实现呢？

```
name = input()
score = data[name] # 从data中获得score, 下节课会讲到
print("%s的成绩是%d" % (name, score))
```



4.3 格式化输出



- % 用法： 整数的输出
- 格式化输出 % 和format

1、整数的输出

%o —— oct 八进制

%d —— dec 十进制

%x —— hex 十六进制

```
>>> print('%o' % 20)
```

```
24
```

```
>>> print('%d' % 20)
```

```
20
```

```
>>> print('%x' % 20)
```

```
14
```

4.3 格式化输出



➤ % 用法：浮点数输出 格式化输出

%f ——保留小数点后面六位有效数字
%.3f, 保留3位小数位

%e ——保留小数点后面六位有效数字，指数形式输出
%.3e, 保留3位小数位，使用科学计数法

%g ——在保证六位有效数字的前提下，使用小数方式，否则使用科学计数法
%.3g, 保留3位有效数字，使用小数或科学计数法

```
>>> print('%f' %  
1.11) # 默认保留6位  
小数  
1.110000  
>>> print('%.1f' %  
1.11) # 取1位小数  
1.1
```

```
>>> print('%e' % 1.11) #  
默认6位小数，用科学计数法  
1.110000e+00  
>>> print('%.3e' % 1.11)  
# 取3位小数，用科学计数法  
1.110e+00
```

```
>>> print('%g' % 1111.1111) # 默认6位  
有效数字  
1111.11  
>>> print('%.7g' % 1111.1111) # 取7位  
有效数字  
1111.111  
>>> print('%.2g' % 1111.1111) # 取2位  
有效数字，自动转换为科学计数法  
1.1e+03
```

6.3 格式化输出



➤ % 用法：浮点数输出 内置 round ()

`round(number[, ndigits])`

参数：

`number` - 这是一个数字表达式。

`ndigits` - 表示从小数点到最后四舍五入的位数。默认值为0。

返回值：

该方法返回x的小数点舍入为n位数后的值。

`round()` 函数只有一个参数，不指定位数的时候，返回一个整数，而且是最靠近的整数，类似于四舍五入，当指定取舍的小数点位数的时候，一般情况也是使用四舍五入的规则，但是碰到.5的情况时，如果要取舍的位数前的小数是奇数则直接舍弃，如果是偶数则向上取舍。

注：“.5”，python2和python3出来的接口有时候是不一样的，尽量避免使用`round()`函数

4.3 格式化输出



```
>>> round(1.1125)  # 四舍五入，不指定位数，取整
1
>>> round(1.1135, 3)  # 取3位小数，由于3为奇数，则向下“舍”
1.113
>>> round(1.1125, 3)  # 取3位小数，由于2为偶数，则向上“入”
1.113
>>> round(1.5)  # 无法理解，python会对数据截断?没有深究
2
>>> round(2.5)  # 无法理解
2
>>> round(1.675, 2)  # 无法理解
1.68
```


4.3 格式化输出



➤ % 用法：字符串输出 字符串输出

%s

%10s——右对齐，占位符10位

%-10s——左对齐，占位符10位

%.2s——截取2位字符串

%10.2s——10位占位符，截取两位字符串

```
>>> print('%s' % 'hello world') # 字符串输出
```

```
hello world
```

```
>>> print('%20s' % 'hello world') # 右对齐，取20位，不够则补位
```

```
hello world
```

```
>>> print('%-20s' % 'hello world') # 左对齐，取20位，不够则补位
```

```
hello world
```

```
>>> print('%.2s' % 'hello world') # 取2位
```

```
he
```

```
>>> print('%10.2s' % 'hello world') # 右对齐，取2位
```

```
he
```

```
>>> print('%-10.2s' % 'hello world') # 左对齐，取2位
```

```
he
```

4.3 格式化输出



➤ % 用法：其他

字符串格式代码

符号	说明
%s	字符串
%c	字符
%d	十进制（整数）
%i	整数
%u	无符号整数
%o	八进制整数
%x	十六进制整数
%X	十六进制整数大写
%e	浮点数格式1
%E	浮点数格式2
%f	浮点数格式3
%g	浮点数格式4
%G	浮点数格式5
%%	文字%

常用转义字符

转义字符	描述
\ (在行尾时)	续行符
\\	反斜杠符号
\'	单引号
\"	双引号
\a	响铃
\b	退格 (Backspace)
\e	转义
\000	空
\n	换行
\v	纵向制表符
\t	横向制表符
\r	回车
\f	换页
\oyy	八进制数yy代表的字符，例如：\o12代表换行
\xyy	十进制数yy代表的字符，例如：\x0a代表换行
\other	其它的字符以普通格式输出

4.3 格式化输出

➤ 格式化输出：用format ()进行格式化输出：

把字符串当成一个模板，通过传入的参数进行格式化，且使用大括号 ‘{}’ 作为特殊字符代替 ‘%’

```
print( “..... {....} .....” .format (<参数表>))
```

<参数序号>	:	<填充 字符>	<对齐符号>	<宽度>	<, >	<. 精度>	<类型>
代表要接收 参数表的第 几个参数， 以0开始，如 果不写，则 都不写，参 数表参数个 数要和{}个 数相同	分隔 符号	用于填 充空白 的单个 字符， 不写默 认为空 格。	^代表居中对齐，<代表 左对齐，>代表右对齐， 有填充字符时必须有对 齐符号，不写数字默认 右对齐，字符串默认左 对齐	设定的输 出宽度， 大于则用 填充字符 填充，小 于或等于 则无效	数字 的千分 位分隔 符，不 写就不 分隔	浮点数的 小数位数 或字符串 的最大输 出长度	整型:b(二进制),o(八进制),d(十进 制),x(十六进制),X(十六进制字母 大写)，不写默认为d 浮点型(默认保留6位小数):f(一般 浮点型),e(科学计数法),%(百分制)， 不写默认为f 字符串型:c(单个字符或ASCII 码),s(字符串)，不写默认为s

4.3 格式化输出



➤ 格式化输出 (Formatted output):

```
>>> print("{:=^16.4s}".format("python"))
=====pyth=====
>>> print("{:8.2f}is a float.".format(2.782223))
    2.78is a float.
>>> print("I love {:8s} !".format("python"))
I love python    !
```

```
>>> print("{0:,.2f}  {0:,.2e}  {0:f}".format(31415.9265))
31,415.93  3.14e+04  31415.926500
>>> print("{1:<10c} and {0:10}".format(65,97))
a           and           65
```

➤ 计算 $1+2+3+\cdots+m$, 此次 m 通过 键盘输入确定

```
m = int(input())  
Sum = 0  
for i in range(1, m + 1):  
    Sum += i  
print("sum = {}".format(Sum))
```

本题目要求计算下列分段函数 $f(x)$ 的值:

$$y = f(x) = \begin{cases} \frac{1}{x} & x \neq 0 \\ 0 & x = 0 \end{cases}$$

输入格式:

输入在一行中给出实数 x 。

输出格式:

在一行中按“ $f(x) = \text{result}$ ”的格式输出，其中 x 与 result 都保留一位小数。

输入样例1:

10

输出样例1:

$f(10.0) = 0.1$

```
x = float(input())
if x:
    y = 1.0 / x
else:
    y = 0
print("f({:.1f}) = {:.1f}".format(x, y))
```

[< 返回](#)

第2章-10 输出华氏-摄氏温度转换表 (15point(s))

输入2个正整数 `lower` 和 `upper` (`lower` ≤ `upper` ≤ 100)，请输出一张取值范围为[`lower` , `upper`]、且每次增加2华氏度的华氏-摄氏温度转换表。

温度转换的计算公式： $C = 5 \times (F - 32) / 9$ ，其中： C 表示摄氏温度， F 表示华氏温度。

输入格式:

在一行中输入2个整数，分别表示 `lower` 和 `upper` 的值，中间用空格分开。

输出格式:

第一行输出: "fahr celsius"

接着每行输出一个华氏温度fahr（整型）与一个摄氏温度celsius（占据6个字符宽度，靠右对齐，保留1位小数

若输入的范围不合法，则输出"Invalid."。

输入样例1:

```
32 35
```

输出样例1:

```
fahr celsius
32    0.0
34    1.1
```

```
lower, upper = map(int, input().split())
if lower > upper or upper > 100:
    print("Invalid.")
else:
    print("fahr celsius")
    for i in range(lower, upper + 1, 2):
        fahr = i
        celsius = 5 * (i - 32) / 9
        print("%d%6.1f"%(fahr, celsius))
```